

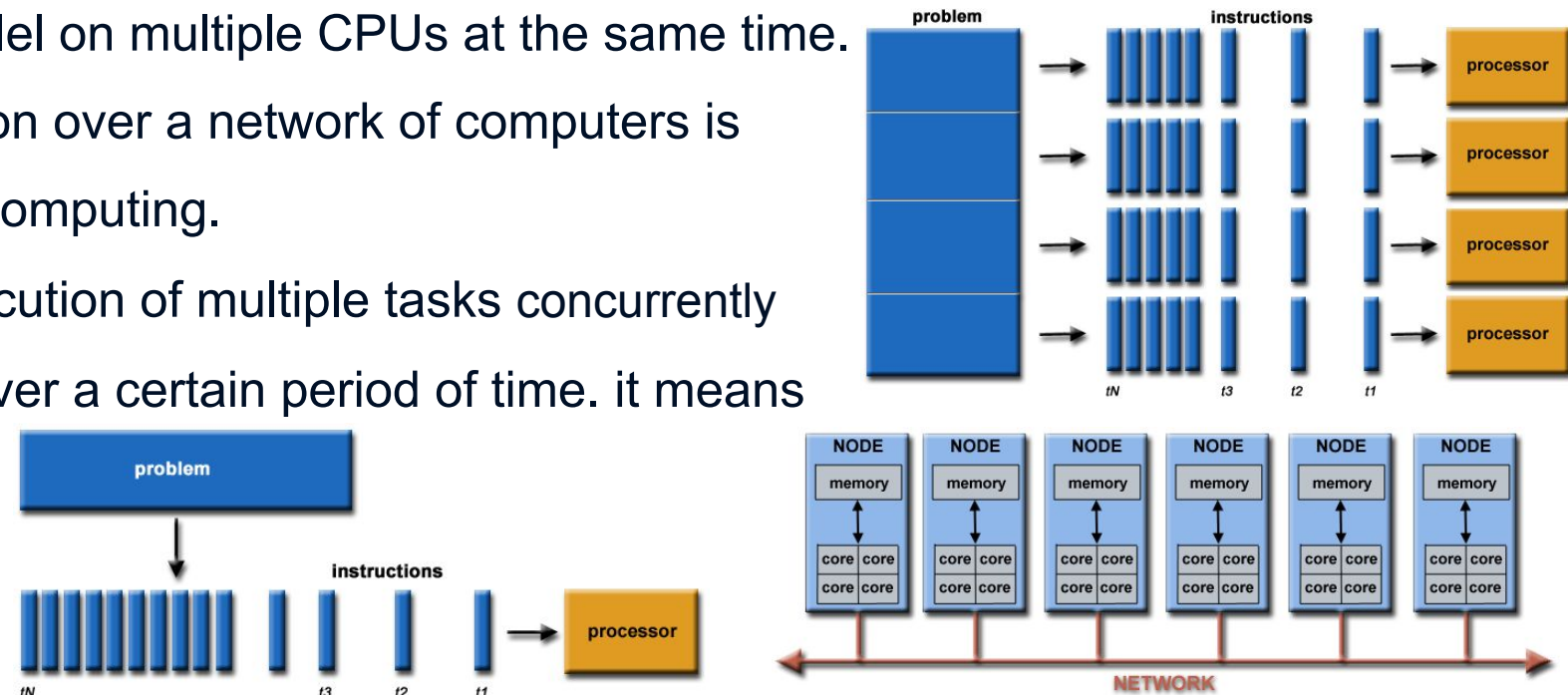


# Parallelism

C++ Language

# Parallelism

- ❖ A **task** is a unit of work that a processor can dispatch, execute, and suspend
- ❖ **Parallelism** means that an application splits its tasks into smaller subtasks which can be processed in parallel on multiple CPUs at the same time.
- ❖ Parallel computation over a network of computers is called distributed computing.
- ❖ **Multitasking** is execution of multiple tasks concurrently on a single CPU over a certain period of time. it means that the CPU time is shared among different tasks through time-slicing. It gives an illusion of parallelism by quickly switching between tasks



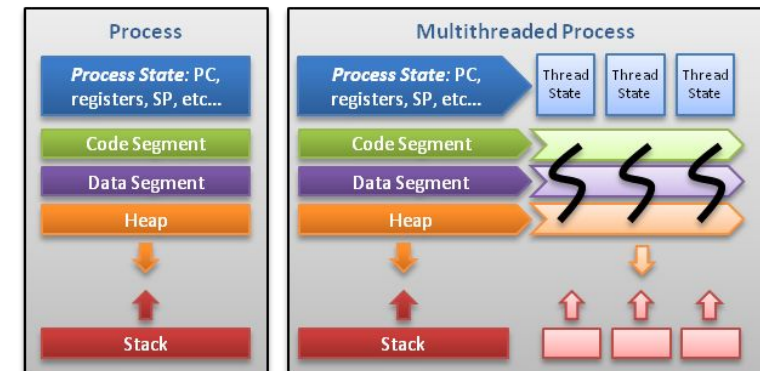
# Parallelism

---

- ❖ Parallel computation has advantages, but it takes resources
  - Spawning threads and forking processes are not free!
- ❖ The architecture that programs run on can facilitate parallelism.
- ❖ In a software we can achieve parallel computation in two ways.
  - Using **multiprocessing** where multiple processes work in parallel
  - Using **multithreading** where multiple threads work concurrently
- ❖ Usually multiprocessing is used where we want to make scalable softwares.
  - Do same thing on different data. Like a web server
  - By forking processes we can achieve multiprocessing
- ❖ Usually multithreading is used where we need to divide a task into different smaller tasks and the smaller tasks are executed concurrently.

# Parallelism

- ❖ CPU sockets are divided into cores and cores are divided into threads.
  - Numbers of CPUs: “socket \* cores \* threads”. Using `lscpu` you can find your cpu info.
- ❖ A **process** is an instance of a program that is being executed by one or more threads.
  - Processes don't share memory space.
- ❖ A **thread** is a sequence of instructions within a program that can be executed independently (like a worker)
  - Threads are execution units that share memory space
  - A thread has its own context: Information to allow a thread to be scheduled independently; such as the stack, registers and thread-specific data
- ❖ There are two main concerns in multiprocessing and multithreading
  - Race conditions, data race and protecting shared resources
  - Synchronization, inter-process and inter-thread communication



# Multithreading - C++ STL thread

---

- ❖ Portable Operating System Interface for Unix (POSIX) is an IEEE standard that helps all Unix and Unix-like operating systems to communicate with a single interface.
- ❖ Since C++11, C++ supports threads via the C++ thread and concurrent library.
- ❖ **pthread** is a POSIX library. The standard C++ thread library is based on pthread
- ❖ If the compiler does not link pthread, link pthread to your program. I.e. **-lpthread**
- ❖ The [std::thread](#) class represents a single thread of execution.
- ❖ Threads are scheduled/executed immediately upon construction
  - A task can be passed to the constructor of std::thread
    - It's possible to pass arguments to the task
  - A task is a callable object like a function, a lambda, a functor, a member function and etc.
  - A thread is movable using std::move

# Multithreading - C++ STL thread

---

- ❖ Threads shall either be **joined** or **detached**; otherwise the program crashes.
  - **detach()** permits the thread to execute independently from the thread handle.
  - **join()** waits on the thread to finish its execution
- ❖ Functions to manage the current thread
  - [std::this\\_thread::get\\_id](#) returns the thread id of the current thread
  - [std::this\\_thread::yield](#) is used to ask the OS to reschedule the current thread
  - [std::this\\_thread::sleep\\_for](#) stops execution of the current thread for a specified time duration
  - [std::this\\_thread::sleep\\_until](#) stops execution of the current thread until a specified time point
- ❖ The [thread\\_local](#) keyword is used to create objects with thread storage duration
  - The storage for the object is allocated when the thread begins and deallocated when the thread ends. Each thread has its own instance of the object. `thread_local` can appear together with `static` or `extern` to adjust linkage. E.g. `static thread_local int var{0}` in a thread function which is executed by multiple threads, for each thread the compiler allocates memory for **var**, unlike normal [static](#) variables.



# Multithreading - C++ STL thread

```
#include <chrono>
#include <thread>
#include <iostream>

void func1(int n) { ++n; std::cout << "Thread 1 executing\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

void func2(int &n) { ++n; std::cout << "Thread 2 executing\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

struct A { int n{0}; void func(void) { ++n;
    std::cout << "Thread 4 executing\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}
};

struct B { int n{0}; void operator()() {
    ++n; std::cout << "Thread 5 executing\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}
};
```

```
int main(void) {
    A a; B b; int n{0};

    std::thread thrd0; // Not an execution thread
    std::thread thrd1{func1, n}; // pass by value
    std::thread thrd2{func2, std::ref(n)}; // pass by reference
    std::thread thrd3{[&] { ++n; std::cout << "Thread 3 executing\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }};

    std::thread thrd4{&A::func, &a};
    std::thread thrd5{std::ref(b)}; // or thrd5(b);

    thrd1.join(); thrd2.join();
    thrd3.join(); thrd4.join();
    thrd5.join();

    std::cout << "Final value of n is " << n << std::endl;
    std::cout << "Final value of f.n (a::n) is " << a.n << std::endl;
    std::cout << "Final value of b.n (b::n) is " << b.n << std::endl;

    return 0;
}
```

# Multithreading - C++ STL thread

---

- ❖ Shared resources can be protected and threads can be synchronize by means of
  - Atomic objects which are free of data race (lock free)
  - Mutexes to prevent multiple threads from simultaneously accessing shared resources
  - Condition variables which allow multiple threads to communicate to each other
- ❖ An atomic operation on an object is an operation which does not get interrupted
- ❖ To use the atomics features, include the `<atomic>` header
- ❖ [`std::atomic`](#) is a template class with a large number of predefined typedefs
  - For example, `std::atomic_bool` which is `std::atomic<bool>`
  - [`std::atomic\_flag`](#) is an atomic boolean type. Unlike `std::atomic<bool>` and all specializations of `std::atomic`, it is guaranteed to be lock-free.
  - Not all operations are supported on atomic objects. Look at the supported [operators](#) and [operations](#)
- ❖ Atomic objects may be used for inter-thread synchronization and protecting shared objects



# Multithreading - C++ STL mutex

- ❖ Atomic operations are generally read, write, read-modify-write. E.g. `count++`;
- ❖ The `<mutex>` header contains multiple types of mutexes and locks. The mutex type is the most commonly used type and provides the lock/unlock mechanism
- ❖ Types of provided mutex by the STL in `<mutex>`
  - [`std::mutex`](#): provides basic mutual exclusion facility
    - [`std::timed\_mutex`](#): with a timeout
  - [`std::recursive\_mutex`](#): mutex which can be locked recursively by the same thread
    - [`std::recursive\_timed\_mutex`](#): with a timeout
  - [`std::shared\_mutex`](#): provides shared mutual exclusion
    - [`std::shared\_timed\_mutex`](#): with a timeout

```
#include <thread>
#include <atomic>
#include <iostream>

static std::atomic<int> count{0};

void increment(void) {
    count++; // This operation is atomic
}

int main(void) {
    std::thread t1{increment}; std::thread t2{increment};
    std::thread t3{increment}; std::thread t4{increment};
    std::thread t5{increment};

    t1.join(); t2.join(); t3.join();
    t4.join(); t5.join();

    std::cout << "Count value:" << count << std::endl;
    return 0;
}
```

# Multithreading - C++ STL mutex

---

- ❖ Usually a mutex is not used directly.
- ❖ There are different managers to manage a mutex in a safer way
  - Lock guard ([`std::lock\_guard`](#))
    - A mutex wrapper which implements a scope-based mutex ownership
  - Scoped lock ([`std::scoped\_lock`](#))
    - A mutex wrapper which is used to own multiple mutexes for the duration of a scoped block.
    - Offers a replacement for `lock_guard` that provides the ability to lock multiple mutexes (C++17)
  - Unique lock ([`std::unique\_lock`](#))
    - Implements a movable mutex ownership wrapper.
    - Unlike `lock_guard` and `scoped_lock`, it is possible to lock and unlock
  - Shared lock ([`std::shared\_lock`](#))
    - Implements movable shared mutex ownership wrapper.
    - To wait on a shared mutex in the shared ownership mode

# Multithreading - C++ STL mutex

- ❖ Deadlock occurs when two or more threads wait on each other to release resources that they need
- ❖ [Strategies](#) when we create an instance of locks to avoid deadlock.
  - **defer\_lock**: do not acquire ownership of the mutex
    - It cannot be used for lock\_guard and scoped\_lock
  - **try\_to\_lock**: try to acquire ownership of the mutex without blocking
    - It cannot be used for lock\_guard and scoped\_lock
  - **adopt\_lock**: the calling thread already has ownership of the mutex
    - It can be used for guard, scoped, unique and shared locks

```
int main(void) {
    for (size_t i = 0; i < 500; i++) {
        Account myAccount{100}; Account yourAccount{50};
        std::thread thrd1{transfer, std::ref(myAccount), std::ref(yourAccount), 10};
        std::thread thrd2{transfer, std::ref(yourAccount), std::ref(myAccount), 5};
        thrd1.join(); thrd2.join();
        std::cout << std::endl << "Run: " << (i + 1) << std::endl;
        std::cout << "myAccount.balance = " << myAccount.balance << std::endl;
        std::cout << "yourAccount.balance = " << yourAccount.balance << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
    return 0;
}
```

```
#include <mutex>
#include <thread>
#include <iostream>

struct Account { int balance; std::mutex mtx;
    explicit Account(int _balance) : balance{_balance} {}
};

void transfer(Account &from, Account &to, int amount) {
    // Avoid deadlock in case of self transfer
    if (&from != &to) {
        // thrd1 tries to lock myAccount.mtx and thrd2 tries to lock yourAccount.mtx
        std::lock_guard from_lock{from.mtx};

        // thrd1 tries to lock yourAccount.mtx and thrd2 tries to lock myAccount.mtx
        std::lock_guard to_lock{to.mtx};

        from.balance -= amount; to.balance += amount;
    }
}
```

# Multithreading - C++ STL Condition Variables

---

- ❖ A condition variable is a synchronization primitive that allows multiple threads to communicate to each other. A condition variable is always associated with a mutex.
- ❖ The [`std::condition\_variable`](#) class has been defined in the `<condition_variable>` header
- ❖ `std::condition_variable_any` provides a condition variable that works with `std::shared_lock`.
- ❖ **`std::condition_variable`** works only with **`std::unique_lock<std::mutex>`**.
  - Acquire a `std::mutex` using a `std::unique_lock`
  - Modify the shared variable while the lock is owned
  - Call `notify_one` or `notify_all` on the `std::condition_variable` (can be done after releasing the lock)
- ❖ `notify_one()` is used to notify a waiting thread and `notify_all()` to notify all waiting threads
- ❖ The `wait` function blocks the current thread until the condition variable is notified
  - `void wait(std::unique_lock<std::mutex> &lock);` and `while (!stop_waiting()) { wait(lock); }`
  - `void wait(std::unique_lock<std::mutex> &lock, Predicate stop_waiting);`
    - The predicate function shall be like: `bool pred(void);`

# Multithreading - C++ STL future

---

- ❖ The standard library provides high level facilities to obtain values that are returned and to catch exceptions that are thrown by asynchronous tasks (i.e. functions launched in separate threads). To use async multitasking include `<future>`.
- ❖ An asynchronous operation is created via [`std::async`](#), [`std::packaged\_task`](#), or [`std::promise`](#)
- ❖ The class template [`std::future`](#) provides a mechanism to access results of asynchronous operations
- ❖ The creator of an asynchronous operation can query, wait for, or extract a value from the `std::future`. If the result is not ready, `std::future` blocks!
- ❖ When an asynchronous operation is ready to send its result to the creator, it can modify the shared state (e.g. `std::promise::set_value`) that is linked to the creator's `std::future`.

# Multithreading - C++ STL future

- ❖ `std::async` runs a function asynchronously (potentially in a new thread) and returns a `std::future` that will hold the result

```
#include <chrono>
#include <future>
#include <thread>
#include <iostream>

int main(void) {
    std::cout << std::endl;

    auto a = std::async(std::launch::async, [] {
        std::this_thread::sleep_for(std::chrono::seconds(2));
        std::cout << "first thread" << std::endl; });

    auto b = std::async(std::launch::async, [] {
        std::this_thread::sleep_for(std::chrono::seconds(1));
        std::cout << "second thread" << std::endl; });

    std::cout << "main thread" << std::endl;

    return 0;
}
```

```
async( std::launch policy, Function&& f, Args&&... args );
```

| Constant                           | Explanation                                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>std::launch::async</code>    | a new thread is launched to execute the task asynchronously                                         |
| <code>std::launch::deferred</code> | the task is executed on the calling thread the first time its result is requested (lazy evaluation) |

<https://en.cppreference.com/w/cpp/thread/async>

```
#include <future>
#include <chrono>
#include <iostream>

bool is_prime(int x) { bool result{true};
    for (int i = 2; i < x; ++i) { if (x % i == 0) { result = false; } } return result;
}

int main(void) {
    std::future<bool> fut = std::async(std::launch::async, is_prime, 444444443);
    std::cout << "Checking, please wait"; std::chrono::milliseconds span(100);

    while (fut.wait_for(span) == std::future_status::timeout)
        { std::cout << '.' << std::flush; }

    bool x = fut.get();

    std::cout << std::endl << "444444443 " << (x ? "is" : "is not") << " prime." << std::endl;

    return 0;
}
```



# Multithreading - C++ STL future

- ❖ A `std::packaged_task` is a wrapper for any callable target (function, bind, lambda, or other function object), so that it can be invoked asynchronously.
- ❖ It allows for asynchronous execution with the result available in a future object.
- ❖ It is similar to `std::function`, but automatically transfers its results to a future object.

```
#include <cmath>
#include <thread>
#include <future>
#include <iostream>
#include <functional>

int func(int x, int y) { return std::pow(x, y); }

void task_lambda(void) {
    std::packaged_task<int(int, int)> task{[](int a, int b)
                                           { return std::pow(a, b); }};
    std::future<int> future = task.get_future();
    task(2, 9);
    std::cout << "task_lambda: " << future.get() << std::endl;
}
```

```
void task_bind(void) {
    std::packaged_task<int()> task{std::bind(func, 2, 11)};
    std::future<int> future = task.get_future();
    task();
    std::cout << "task_bind : " << future.get() << std::endl;
}

void task_thread(void) {
    std::packaged_task<int(int, int)> task{func};
    std::future<int> future = task.get_future();
    std::thread task_td(std::move(task), 2, 10);
    task_td.join();
    std::cout << "task_thread: " << future.get() << std::endl;
}
```

```
int main(void) {
    task_bind();
    task_lambda();
    task_thread();
    return 0;
}
```

# Multithreading - C++ STL future

---

- ❖ The class template [`std::promise`](#) provides a facility to store a value or an exception that is later acquired asynchronously via a `std::future` object created by the `std::promise` object.
  - non-void specialization, used to communicate objects between threads
  - void specialization, used to communicate stateless events
- ❖ Promises allow us to transfer states between threads.
- ❖ Note that a `std::promise` object is meant to be used only once.
- ❖ Each promise is associated with a shared state, which contains some **state information** and a **result** which may not be evaluated yet, evaluated to a value (possibly void) or evaluated to an exception.

# Multithreading - C++ STL future

❖ A promise shared state may be:

- **ready**: the promise stores the result or the exception in the shared state. Marks the state ready and unblocks any thread waiting on the future associated with the shared state.
- **release**: the promise gives up its reference to the shared state. If this was the last state, the shared state is destroyed. Unless this was a shared state created by `std::async` which is not yet ready, this operation does not block.
- **abandon**: the promise stores the exception of type [`std::future\_error`](#) with an error code

```
#include <chrono>
#include <thread>
#include <future>

void do_work(std::promise<void> barrier) {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    barrier.set_value();
}
```

```
int main(void) {
    // Using promise<void> to signal state between main and work_thread threads.
    std::promise<void> barrier;
    std::future<void> barrier_future = barrier.get_future();
    std::thread work_thread(do_work, std::move(barrier));
    barrier_future.wait(); work_thread.join();
    return 0;
}
```

# Multithreading - C++ STL future

```
#include <thread>
#include <future>
#include <iostream>

int main(void) {
    std::promise<int> promise;
    std::future<int> future = promise.get_future();
    std::thread thrd([&promise] {
        try {
            // Code that may throw
            throw std::runtime_error("Example");
            promise.set_value(10);
        } catch(...) {
            try {
                // Store anything thrown in the promise
                promise.set_exception(std::current_exception());
                // or throw a custom exception instead
                // promise.set_exception(std::make_exception_ptr(MyException("mine")));
            } catch(...) {} // set_exception() may throw too
        }
    });
    try { std::cout << future.get(); }
    catch (const std::exception &e)
    { std::cout << "Exception from the thread: " << e.what() << std::endl; }
    thrd.join(); return 0;
}
```

```
#include <thread>
#include <future>
#include <iostream>
#include <functional>

void print_int(std::future<int> &fut) {
    int x = fut.get(); std::cout << "value: " << x << std::endl;
}

int main(void) {
    std::promise<int> prom;
    std::future<int> fut = prom.get_future();
    std::thread th1(print_int, std::ref(fut));
    std::this_thread::sleep_for(std::chrono::seconds(1));
    prom.set_value(10);
    th1.join();
    return 0;
}
```

# Multithreading - C++ STL future

---

```
#include <vector>
#include <thread>
#include <future>
#include <chrono>
#include <numeric>
#include <iostream>

void accumulate(std::vector<int>::iterator first, std::vector<int>::iterator last, std::promise<int> accumulate_promise) {
    int sum = std::accumulate(first, last, 0); accumulate_promise.set_value(sum); // Notify future
}

int main(void) {
    // Using promise<int> to transmit a result between threads.
    std::promise<int> accumulate_promise; std::vector<int> numbers = {1, 2, 3, 4, 5, 6};
    std::future<int> accumulate_future = accumulate_promise.get_future();
    std::thread work_thread(accumulate, numbers.begin(), numbers.end(), std::move(accumulate_promise));

    // future::get() will wait until the future has a valid result and retrieves it.
    // Calling wait() before get() is not needed. accumulate_future.wait(); // wait for result
    std::cout << "result=" << accumulate_future.get() << std::endl;

    work_thread.join(); // wait for thread completion
    return 0;
}
```

# Multithreading - C++ STL future

---

- ❖ **std::future** is only movable, thus cannot be shared amongst threads
- ❖ The class template [std::shared\\_future](#) provides a mechanism to access the result of asynchronous operations, similar to `std::future`, except that multiple threads are allowed to wait on the same shared state.
- ❖ Unlike `std::future`, `std::shared_future` is copyable and multiple shared future objects may refer to the same shared state.
- ❖ Access to the same shared state from multiple threads is safe if each thread does it through its own copy of a `shared_future` object.
- ❖ A `shared_future` may be used to signal multiple threads simultaneously
  - Similar to `std::condition_variable::notify_all()`

# Multithreading - C++ STL future

---

```
#include <future>
#include <chrono>
#include <iostream>

int main(void) {
    std::promise<void> ready_promise, ready_promise_async;
    std::shared_future<void> ready_future(ready_promise.get_future());
    std::chrono::time_point<std::chrono::high_resolution_clock> start;

    /* ready_future is captured by value. I.e. A copy of ready_future */
    auto task = [&, ready_future]() -> double {
        ready_promise_async.set_value();
        ready_future.wait(); /* waits for the signal from main() */
        std::chrono::duration<double, std::milli> duration{std::chrono::high_resolution_clock::now() - start};
        return duration.count();
    };

    auto future = ready_promise_async.get_future();
    auto result = std::async(std::launch::async, task);

    future.wait(); // wait for the thread to become ready
    start = std::chrono::high_resolution_clock::now(); // the thread is ready, start the clock
    ready_promise.set_value(); // signal the thread to go
    std::cout << "Thread received the signal " << result.get() << " ms after start" << std::endl;

    return 0;
}
```



# C++ Language

---

## ❖ Some useful links

- [C++ Reference](#)
- [Standard C++ Library Reference](#)
- [C++ Tutorial](#)
- [C++ Language](#)
- [C++ Tutorial](#)
- [C++ Full Course For Beginners \(Learn C++ in 10 hours\)](#)
- [C++ Tutorial for Beginners - Full Course](#)
- [C++ Tutorial 2021](#)
- [C++ Programming Tutorials Playlist](#)
- [C++ Multithreading](#)
- [C++ Tutorial: Threads](#)
- [C++ Concurrency](#)